

2025학년도 학사학위논문 (종합설계보고서)

지도교수 한 동 현 교수님

CNN 컨볼루션 연산을 위한
준-범용 FPGA 가속기 설계 및 구현

중앙대학교 전자전기공학부

2025. 11.

목 차

요약

I. 서론	5
1. 연구 배경 및 필요성	5
2. 연구 목적 및 목표	5
3. 연구 범위 및 구성	6
II. 관련 이론 및 요구사항	7
1. CNN 컨볼루션 연산 원리	7
2. 정수형 양자화 이론	7
3. Zynq SoC 및 시스템 요구사항	8
(1) Zynq SoC 개요	8
(2) AXI 인터페이스 구조	8
(3) 시스템 요구사항 요약	8
(4) 개발 환경	9
III. 시스템 설계 및 구현	10
1. 전체 시스템 구조	10
2. 데이터 흐름 및 동작 시퀀스	11
(1) 파라미터 설정 단계	11
(2) 입력 데이터 전송 단계	11
(3) 연산 단계	11
(4) 출력 데이터 전송 단계	11
3. 하드웨어 모듈 설계	12
3.3.1 Shift Buffer	12
3.3.2 MAC9 병렬 유닛	13
3.3.3 BRAM 및 FIFO 메모리	14
3.3.4 FSM 제어 및 동작	16
3.3.5 가변 파라미터에 대응하기 위한 구조적 설계 아이디어	17
(1) 입력 이미지 크기에 대한 대응	17
(2) 패딩과 스트라이드에 대한 대응	17
(3) 입력 및 출력 채널에 대한 대응	17
(4) 정리 및 의의	18
4. IP 생성 및 SoC 통합	19
(1) IP 생성	19
(2) 블록 디자인 구성	19
(3) 합성, 구현 및 비트스트림 생성	19
5. Vivado 합성 및 자원 사용 결과	20
IV. 실험 및 결과	21
1. 실험 개요	21

2. 실험 환경	21
3. 실험 1 - RTL 시뮬레이션 기초 동작 검증	22
4. 실험 2 - 랜덤 정수 데이터 기반 기능 및 성능 검증	23
5. 실험 3 - 양자화 기반 Simple 모델 추론 검증	24
6. 실험 4 - 양자화 기반 상용 모델 추론 검증	25
V. 결론	27
참고 문헌	28

Abstract

본 연구에서는 Zynq-7000 SoC 기반 FPGA(Arty Z7-20) 상에 3*3 컨볼루션 연산 전용 CNN 가속기 IP를 설계하고 검증하였다. 제안된 가속기는 기존의 특정 모델 구조에만 활용가능한 하드코딩형 CNN 가속기와는 달리 다양한 입력 크기, 채널 수, 스트라이드, 패딩 값에 유연하게 대응할 수 있는 준-범용(Semi-General Purpose) CNN 가속기로 설계되었으며, Shift Buffer 기반 입력 윈도우 구성, 16개의 MAC9 병렬 연산 유닛, BRAM 및 FIFO 누적 구조를 통해 DRAM 접근을 최소화하고 데이터 재사용률을 극대화하였다.

MAC9 유닛은 3단 파이프라인 구조로 구성되어 매 클럭마다 새로운 필터의 컨볼루션 결과를 생성할 수 있으며, BRAM은 필터 및 바이어스 데이터를 저장하는 내부 메모리로 사용되고, FIFO는 출력 피쳐맵 누적을 담당하여 BRAM의 사용량을 최소화하였다. 또한 RTL 설계 중 parameter를 활용하여, 동일한 RTL 코드를 다른 규격의 컨볼루션 연산을 목적으로 수정하기 용이하도록 설계하였다.

시스템 제어는 PYNQ 기반 Python 드라이버 코드를 통해 수행되었다. Python 레벨에서 CNN의 하이퍼 파라미터(입력 크기, 채널 수, 스트라이드, 패딩 등)를 AXI-Lite 레지스터로 전송하고, DMA를 이용해 입력, 가중치, 출력 데이터를 AXI-Stream 방식으로 스트리밍 전송함으로써, Python 코드가 단순한 테스트 스크립트를 넘어 하드웨어 드라이버 역할을 수행하도록 하였다. 이를 통해 PS와 PL이 유기적으로 동작하는 HW/SW SoC 구조를 완성하였으며, Vivado IP Packager를 이용한 AXI 표준 IP 패키징으로 재사용성과 확장성을 확보하였다.

Vivado 합성 및 구현 결과, 제안된 IP는 Arty Z7-20 보드의 제한된 자원(LUT, FF, BRAM, DSP) 내에서 안정적으로 구현되었으며, BRAM 및 FIFO 구조의 효율적 혼용으로 메모리 자원 사용량을 절감, 100 MHz 동작 주파수에서 안정적 타이밍 여유 확보를 달성하였다.

실험은 세 단계로 수행되었다.

첫째, RTL 레벨에서 설계의 의도대로 컨볼루션 연산이 이루어지는지 테스트벤치 시뮬레이션을 진행하였다.

둘째, 랜덤 정수형 입력, 가중치 데이터를 이용한 기능 및 성능 검증을 통해, FPGA 연산 결과가 CPU(ARMv7I 기반) 연산 결과와 완벽히 일치하며 연산 성능 또한 우수함을 확인하였다.

셋째, CIFAR-10 데이터셋으로 학습된 Simple VGG 모델을 이용하여 전 과정 CPU 추론 결과와 CPU + 정수형 양자화 FPGA 가속 하이브리드 추론 결과를 비교하였다. 그 결과, 1000개의 테스트 셋에 대해 하이브리드 추론은 약간의 정확도 손실(0.1%p)만을 보이며, CPU 대비 12.71배의 속도 향상을 달성하였다.

넷째, 동일한 방식으로 VGG16 상용 모델을 파인튜닝하여 검증한 결과, 100개의 테스트 셋에 대해 하이브리드 추론은 CPU 단독 추론 대비 2.0%p의 정확도 손실만을 보이며, 16.88배의 속도 향상을 달성하였다.

결론적으로, 본 연구는 특정 네트워크나 커널 구조에 종속되지 않고, 다양한 CNN 연산에 공통적으로 적용 가능한 준-범용 컨볼루션 연산기 구조를 FPGA 상에 성공적으로 구현하였다. 정수형 양자화 기반의 경량 구조를 통해 연산 자원과 전력 소모를 최소화하면서도, Python을 이용한 제어를 통해 CNN의 주요 연산 블록인 컨볼루션 레이어를 효율적으로 가속할 수 있는 하드웨어 형태로 구축하였다. 본 설계는 임베디드 환경에서 범용적으로 적용 가능한 CNN 컨볼루션 가속기 구조의 가능성을 제시하며, 향후 다양한 네트워크 및 SoC 기반 표준화된 딥러닝 하드웨어 프레임워크로 확장될 수 있다.

I. 서론

1. 연구 배경 및 필요성

최근 딥러닝의 급속한 발전으로 합성곱 신경망(CNN, Convolutional Neural Network)은 영상 인식, 물체 탐지, 분류 등 다양한 분야에서 핵심 알고리즘으로 자리 잡았다. 그러나 CNN의 핵심 연산인 컨볼루션은 연산량이 매우 많고, 특히 임베디드 환경에서는 CPU만으로 실시간 처리가 어렵다.

이 문제를 해결하기 위해 GPU, TPU, NPU 등 다양한 하드웨어 가속기가 제안되어 왔으나, 이러한 장치들은 고비용, 고전력 구조로 인해 소형 임베디드 시스템에는 적합하지 않다. 이에 반해 FPGA(Field Programmable Gate Array)는 병렬 연산이 가능한 구조와 높은 재구성성을 동시에 갖추고 있어, CNN의 컨볼루션 연산을 저전력 환경에서 효율적으로 수행할 수 있는 플랫폼으로 주목받고 있다. 특히 컨볼루션 연산은 대규모의 MAC(Multiply-Accumulate) 연산으로 구성되므로, 이러한 구조적 특성이 FPGA와 잘 부합한다.

본 연구는 이러한 점에 착안하여, Zynq-7000 SoC 기반 FPGA 상에 CNN의 컨볼루션 연산 전용 하드웨어를 구현하고, 다양한 입력 조건에 대응 가능한 준-범용(Semi-General Purpose) 구조를 제시한다. 여기서 ‘준-범용’이란, 특정 네트워크 모델에 종속되지 않고 입력 크기, 채널 수, 패딩, 스트라이드 조합에 유연하게 대응할 수 있는 구조를 의미한다.

기존 학부 수준의 CNN 가속기는 특정 모델 구조(VGG, LeNet 등)에 맞추어 하드웨어를 고정(hard-coded)하는 방식으로 구현되어, 입력 크기나 채널 수가 달라질 경우 재설계가 필요하다는 문제가 있었다. 본 연구는 이러한 한계를 해결하고자, 다양한 입력 조건 변화에도 동일한 하드웨어로 동작할 수 있는 준-범용 IP 구조를 제안한다. 이를 통해 하나의 IP로 여러 CNN 컨볼루션 레이어를 처리할 수 있게 하였으며, 나아가 PS와 PL을 통합한 HW/SW SoC 시스템을 구축하여, 하드웨어 가속과 상위 제어를 유기적으로 결합하였다. 하드웨어에서는 컨볼루션 연산 유닛(MAC9)를 중심으로 Shift Buffer, BRAM, FIFO 메모리 구조를 RTL 수준에서 설계하였고, 소프트웨어에서는 Python(PYNQ)을 이용하여 CNN 추론 과정을 하드웨어 명령으로 변환하는 제어 드라이버를 구현하였다. 이 통합 구조를 통해 CNN의 연산 병목인 컨볼루션 부분을 FPGA에서 고속 처리하면서도, 상위 제어는 Python 수준에서 간편하게 수행할 수 있도록 하였다.

2. 연구 목적 및 목표

본 연구의 목적은 Zynq-7000 SoC(Arty Z7-20) 환경에서 CNN의 컨볼루션 연산을 효율적으로 가속하기 위한 준-범용 IP를 설계하고, 이를 PYNQ 기반으로 HW/SW 통합 검증하는 것이다.

Zynq SoC는 ARM Cortex-A9 프로세서(PS)와 FPGA(PL)가 단일 칩에 통합된 구조를 가지고 있으며, AXI버스를 통해 상호 연결된다. 이 구조를 활용하면 PS가 전체 제어 및 데이터 전송을 담당하고, PL이 컨볼루션 연산의 병렬화를 수행하는 하이브리드 CNN 시스템을 구성할 수 있다. 본 연구에서는 단순한 IP 설계를 넘어, CNN의 컨볼루션 하이퍼 파라미터(입력 크기, 채널 수, 스트라이드, 패딩 등)를 Python 코드에서 AXI-Lite 레지스터로 전송하고, DMA를 통해 AXI-Stream 방식으로 입력, 출력 데이터를 실시간 전송하는 구조를 구축한다. 즉, Python에서의 명령으로 CNN의 컨볼루션 연산을 가속 수행할 수 있는 완전한 SoC HW-SW 통합 구조를 구현하는 것이 본 연구의 핵심 목표이다.

또한 설계의 효율성을 고려하여, 본 연구에서는 커널 크기를 3*3으로 고정하였다. 이는 다양한 커널 크기를 모두 지원하기 위한 복잡한 하드웨어 제어를 줄이면서도, 대부분의 현대 CNN 구조가 3*3 필터를 표준적으로 사용한다는 점에 근거한 합리적인 선택이다. 이로써 하드웨어 자원을 효율적으로 활용하면서도, 실제 응용 환경에서 높은 범용성을 유지할 수 있는 구조를 구현하였다.

또한 RTL 설계에서는 parameter와 localparam을 체계적으로 활용하여 비트 폭, MAC 유닛

개수 등 구조적 사양을 손쉽게 조정할 수 있도록 하였다. 이를 통해 하드웨어 구조의 확장성과 재사용성을 동시에 확보하였으며, 향후 더 큰 자원량의 FPGA 보드에서 하드웨어의 스펙을 개선하고자 할 때도 최소한의 코드 변경으로 재합성이 가능한 유연한 Convolution IP를 구현하는 것을 목표로 삼았다.

3. 연구 범위 및 구성

본 보고서는 다음과 같은 구성으로 이루어진다.

II장에서는 CNN 컨볼루션 연산의 기본 원리와 정수형 양자화(Quantization) 이론, 그리고 시스템의 기능적, 성능적 요구사항을 설명한다.

III장에서는 제안된 컨볼루션 가속기 IP의 하드웨어 구조를 상세히 기술하며, Shift Buffer, MAC9 병렬 유닛, BRAM/FIFO 메모리, FSM 제어 및 AXI 인터페이스를 중심으로 SoC 통합 설계 과정을 다룬다.

IV장에서는 PYNQ 환경에서 수행한 실험 결과를 제시하고, CPU 대비 처리속도 향상과 정수형 양자화 기반 정확도 분석을 수행한다.

V장에서는 본 연구의 주요 성과를 정리하고, 향후 확장 방향을 제시한다.

II. 관련 이론 및 요구사항

1. CNN 컨볼루션 연산 원리

합성곱 신경망은 입력 데이터로부터 특징을 추출하기 위해 컨볼루션 연산을 반복적으로 수행한다. 컨볼루션 연산은 필터를 입력 이미지의 국소 영역에 적용하여 곱셈-누산 연산을 수행함으로써, 출력 피쳐맵(Feature Map)을 생성한다. 이를 수식으로 표현하면 다음과 같다.

$$O_{x,y,k} = \sum_{i=0}^{M-1N-1C-1} \sum_{j=0}^{M-1N-1C-1} \sum_{c=0}^{M-1N-1C-1} I_{(x+i),(y+j),c} \times W_{i,j,c,k} + B_k$$

여기서,

- $I_{(x+i),(y+j),c}$: 입력 이미지의 픽셀 값
- $W_{i,j,c,k}$: k번째 필터의 가중치 값
- B_k : 바이어스
- $M \times N$: 필터 크기 (본 연구에서는 3×3 사용)
- C : 입력 채널 수
- $O_{x,y,k}$: 출력 피쳐맵의 (x,y) 위치에 대한 결과

이 연산은 각 출력 픽셀마다 다수의 곱셈과 덧셈이 필요하며, 특히 입력 채널 수가 많을수록 연산량이 급격히 증가한다. 따라서 CNN의 컨볼루션 연산을 실시간으로 수행하기 위해서는 병렬화와 파이프라인 구조의 효율적 설계가 필수적이다. 본 연구에서는 입력 이미지를 효율적으로 재사용할 수 있는 Shift Buffer 구조와, 다중 채널 누적을 위한 FIFO 기반 구조, 그리고 16개의 병렬 MAC9 연산 유닛을 이용해 연산 효율을 극대화하였다.

2. 정수형 양자화 이론

딥러닝 모델은 일반적으로 부동소수점 연산을 사용하지만, FPGA에서는 부동소수점 연산이 DSP 자원과 LUT를 과도하게 소모하며, 연산 지연(latency) 또한 커지는 단점이 있다. 이에 따라 CNN 모델의 파라미터와 입력 데이터를 정수형으로 변환하여 연산 효율을 높이는 양자화 기법이 필수적으로 사용된다. 양자화는 실수형 데이터를 고정된 정수 비트 폭으로 스케일링하여 표현하는 과정으로, 본 연구에서는 다음과 같이 정의하였다.

1. 양자화 스케일 계산

$$\text{입력 스케일} : S_x = \frac{\max(|x_f|)}{2^7 - 1} = \frac{\max(|x_f|)}{127}$$

$$\text{가중치 스케일} : S_w = \frac{\max(|W_f|)}{2^7 - 1} = \frac{\max(|W_f|)}{127}$$

$$\text{출력 및 바이어스 스케일} : S_{out} = S_x \times S_w$$

2. 정수 변환

$$\text{입력 (float32} \rightarrow \text{int8)} : x_q = \theta\left(\frac{x_f}{S_x}\right)$$

가중치 (float32 $\rightarrow$ int8) : $W_q = \partial(\frac{W_f}{S_w})$

바이어스 (float32 $\rightarrow$ int32) : $B_q = \partial(\frac{B_f}{S_{out}})$

3. 역양자화

최종 결과 복원 (int32 $\rightarrow$ float32) : $Y_q = Conv(x_q, W_q) + B_q, Y_f \approx S_{out} \times Y_q$

본 연구의 하드웨어는 정수형 연산 전용 구조이므로, 입력 데이터와 필터 가중치는 각각 8비트 (INT8), 바이어스는 32비트(INT32)로 변환하여 처리하였다. 이 조합은 DSP 사용량과 LUT 점유율을 최소화하면서도, 누산 과정에서 오버플로우 없이 충분한 정밀도를 유지할 수 있는 효율적인 비트폭 조합이다. 양자화된 모델의 추론 과정에서는 모든 연산이 정수 기반으로 수행되지만, 최종 출력값은 역스케일링을 통해 부동소수점으로 복원된다. 이러한 양자화-복원 과정을 통해 PyTorch의 부동소수점 추론 흐름과 FPGA의 정수형 연산을 결합시킬 수 있었다.

3. Zynq SoC 및 시스템 요구사항

(1) Zynq SoC 개요

Zynq-7000 SoC는 Processing System(PS)과 Programmable Logic(PL)이 단일 보드 내에서 통합된 SoC 구조를 갖는다.

PS는 ARM Cortex-A9 듀얼코어 프로세서로, SW 제어 및 관리 기능을 담당하며 PL은 FPGA 영역으로, 병렬 연산 및 데이터 처리를 수행한다. 본 연구에서는 PS가 CNN의 상위 제어를 담당하고, PL이 컨볼루션 연산을 수행하도록 역할을 분리하였다. PS와 PL 간의 데이터 교환은 AXI 버스를 통해 이루어진다.

(2) AXI 인터페이스 구조

구분	인터페이스명	주요 역할
제어	AXI4-Lite	하드웨어 IP의 레지스터 제어 및 파라미터 설정
데이터 전송	AXI4-Stream	입력, 출력 데이터의 스트리밍 전송
메모리 접근	AXI4 Memory-Mapped (DMA)	DDR 메모리와 IP 간 데이터 버퍼링

이 구조를 통해 Python(PYNQ) 코드가 하드웨어 내부 레지스터를 직접 제어하고, DMA를 통해 입력 이미지 및 필터 데이터를 스트리밍 방식으로 전송할 수 있다.

(3) 시스템 요구사항 요약

① 기능적 요구사항

필터 크기는 3*3으로 고정한다. 입력 크기는 1*1에서 32*32까지 지원하며, 다채널 입력과 최대 512채널 출력을 처리할 수 있도록 설계한다. 패딩은 0~2 범위 내에서, 스트라이드는 제약 없이 설정할 수 있도록 한다. 입력 데이터는 AXI4-Stream 인터페이스를 통해 수신되어 내부 버퍼를 거쳐 컨볼루션 연산을 수행하도록 한다.

② 정확성 및 성능 요구사항

FPGA에서 수행된 정수형 연산 결과가 CPU 연산 결과와 일치하도록 한다. 동일한 입력 데이터에 대해 CPU 대비 수 배 이상의 처리속도를 확보하도록 설계한다. PyTorch 기반 추론과 비교 시

정확도 손실은 가능한 최소로 한다.

③ 확장성 및 재사용성

RTL 설계 단계에서 parameter와 localparam을 활용하여 비트 폭, MAC 유닛 개수 등 구조적 사양을 손쉽게 조정할 수 있도록 한다. 이를 통해 RTL 코드의 대규모 수정 없이도 하드웨어 구조를 유연하게 개선, 확장할 수 있으며, 향후 더 큰 자원량의 FPGA 보드에서도 최소한의 변경으로 재합성이 가능하도록 한다. 또한 단순한 모듈 단위의 설계를 넘어 Vivado IP Packager를 이용해 독립적인 AXI 인터페이스 기반 IP로 구현함으로써, SoC 환경에서 재사용 가능한 수준의 하드웨어 블록으로 완성한다.

④ HW/SW 통합성

Python(PYNQ)을 이용하여 PS-PL 간 통합 제어 구조를 구현한다. CNN의 컨볼루션 연산 하 이퍼 파라미터는 AXI-Lite 레지스터를 통해 설정하고, DMA를 이용해 입력, 출력 데이터를 전송하도록 한다. 연산 시작 및 완료 신호는 AXI-Lite 제어 레지스터를 통해 관리되도록 설계한다.

⑤ 자원 효율성

Arty Z7-20 보드의 제한된 LUT, FF, BRAM, DSP 자원 내에서 동작하도록 최적화한다.

(4) 개발 환경

항목	내용
FPGA 보드	Digilent Arty Z7-20 (Zynq-7020 SoC)
개발 도구	Vivado 2022.2
임베디드 환경	PYNQ 2.7 (Python 3.8)
실험 환경	Jupyter Notebook, PyTorch 1.8.1, Torchvision 0.9.1
인터페이스 구성	AXI4-Lite (제어), AXI4-Stream (데이터)

본 연구는 위 환경을 기반으로 진행되었다. PYNQ 환경은 공식 PYNQ 이미지를 SD 카드에 설치한 뒤, 해당 SD 카드를 보드에 장착하여 부팅함으로써 구성하였다. PyTorch와 Torchvision은 armv7l 아키텍처용 휠(.whl) 파일을 이용해 수동 설치하였다. Vivado 2022.2를 사용하여 RTL 설계, 합성, IP 패키징 및 SoC 통합을 수행하였으며, PYNQ 환경에서는 Jupyter Notebook 기반의 Python 코드를 통해 하드웨어 제어와 데이터 검증을 수행하였다.

Ⅲ. 시스템 설계 및 구현

1. 전체 시스템 구조

본 시스템은 Zynq-7000 SoC(Arty Z7-20)를 기반으로, ARM Cortex-A9 프로세서(PS)와 FPGA 로직(PL)이 단일 보드 내에 통합된 구조를 갖는다. PS 영역은 PYNQ 환경 기반의 사용자 제어, DMA 설정 및 연산 관리를 담당하며, PL 영역에는 본 연구에서 설계한 Convolution IP, 데이터 전송용 AXI DMA, 그리고 각 컴포넌트를 연결하는 AXI Interconnect가 포함된다.

두 영역은 AXI4-Lite 및 AXI4-Stream 인터페이스를 통해 데이터 및 제어 신호를 교환한다.

PS : PYNQ 기반 Python 코드로 CNN 파라미터 제어 및 DMA 동작 관리

DMA : PS-PL 간 데이터 스트리밍 경로

Convolution IP : CNN의 컨볼루션 연산 전용 하드웨어 블록

AXI4-Lite : 하이퍼 파라미터 설정 및 제어 신호 전송

AXI4-Stream : 입력, 필터, 출력 데이터 스트리밍

이 구조는 PS와 PL이 병렬적으로 동작하면서도 상호 제어가 가능하도록 구성되어, CNN의 컨볼루션 연산을 실시간으로 수행할 수 있는 하드웨어-소프트웨어 통합 SoC 시스템을 형성한다.

2. 데이터 흐름 및 동작 시퀀스

본 시스템의 연산 절차는 PS와 PL(FPGA)사이의 협조적 데이터 흐름으로 이루어진다. 그 구조를 단계별로 정리하면 다음과 같다.

(1) 파라미터 설정 단계

Python(PYNQ) 드라이버는 AXI-Lite를 통해 아래 레지스터 맵에 파라미터를 기록/조회한다.

주소 오프셋	파라미터	역할 / 기록 값
0x00	IMAGE_WIDTH	입력 이미지의 크기
0x04	INPUT_CHANNEL	입력 채널 수 C
0x08	OUTPUT_CHANNEL	출력 채널 수 K
0x0C	STRIDE	스트라이드 값
0x10	PADDING	패딩 값
0x14	START	연산 시작 트리거
0x18	EOC_STATUS	연산 종료 상태

(2) 입력 데이터 전송 단계

PS는 DMA를 통해 DDR 메모리에 저장된 입력 이미지와 필터 데이터를 AXI4-Stream 인터페이스를 통해 PL 내부로 전송한다. 데이터는 TVALID/TREADY신호를 통해 스트리밍 방식으로 전송되며, 입력 이미지는 Shift Buffer에 저장되고 필터, 바이어스 데이터는 BRAM에 기록된다. 입력 데이터는 본 연구에서 의도한 연산 규칙과 일치하도록 입력채널 → 행 → 열순서의 1차원 형태로 전송되어야 하며, 필터 데이터는 입력채널 → 출력채널 → 행 → 열순서로 모든 필터 가중치를 일렬화한 뒤, 마지막에 각 필터의 바이어스 데이터를 순차적으로 이어 전송해야 한다.

(3) 연산 단계

컨볼루션 IP는 Shift Buffer를 기반으로 MAC9 병렬 유닛내 적절한 3*3 이미지 윈도우를 구성하며 연산을 수행한다. 입력 채널이 다중인 경우에는 채널별 결과가 FIFO를 통해 순차적으로 누적되어 최종 출력 채널의 피쳐맵이 완성된다. 연산 과정에서는 입력 크기, 채널 수, 패딩, 스트라이드 등 다양한 조합을 제어 로직이 인식하고 이에 맞게 동작하도록 설계되어, 어떤 파라미터 조건에서도 안정적으로 컨볼루션 연산을 수행할 수 있는 유연한 구조를 구현하였다.

(4) 출력 데이터 전송 단계

연산이 완료된 결과값은 AXI4-Stream을 통해 DMA로 전송되며, DMA는 이를 DDR 메모리에 순차적으로 저장한다. 이때 출력 데이터는 행 → 열 → 출력채널 순서로 전송되며, 소프트웨어(Python)에서는 이를 출력채널 → 행 → 열형태로 다시 transpose하여 PyTorch의 텐서 구조와 일치시키도록 처리하여야 한다. 이후 PS는 저장된 결과를 읽어 CPU 기반 연산 결과와 비교하거나 PyTorch 후속 추론 단계로 전달한다. 이와 같은 데이터 흐름은 AXI-Lite(제어)와 AXI-Stream(데이터)의 역할이 명확히 구분된 구조에서 동작한다. 또한 AXI-Stream의 Backpressure 메커니즘(TREADY/TVALID)을 활용하여, 데이터 유실 없이 안정적인 스트리밍 전송이 가능하도록 설계되었다.

3. 하드웨어 모듈 설계

제안된 Convolution IP는 컨볼루션 연산을 효율적으로 수행하기 위해 다음과 같은 주요 하위 구조로 구성된다.

Shift Buffer : 입력 데이터 임시 저장

MAC9 병렬 유닛 : 3*3 윈도우 구성 및 컨볼루션 연산 수행

BRAM 및 FIFO 메모리 : 필터, 바이어스 저장 및 출력 피처맵 누적

FSM : 전체 연산 순서 관리, 제어

AXI 인터페이스 : 외부 PS 및 DMA와 데이터, 제어 신호 교환

3.3.1 Shift Buffer

Shift Buffer는 입력 스트림으로 유입되는 데이터를 임시 저장하며 적합한 3*3 컨볼루션 윈도우를 형성할 수 있도록 돕는 역할을 한다. 입력 데이터는 AXI4-Stream을 통해 1개 단위로 순차적으로 입력되며, FSM의 제어에 따라 매 입력마다 시프트 동작을 수행한다.

① 설계 의도

이미지 버퍼의 데이터를 MAC9 유닛으로 전달하는 과정에서, 동적 인덱싱 방식은 MUX 합성과 함께 구조가 복잡하고 가변 입력 크기, 패딩, 스트라이드를 처리하기 어렵다. 이러한 문제를 해결하기 위해 이미지 버퍼 자체가 시프트되어 데이터가 이동하고, 지정된 레지스터만 MAC9 유닛과 연결해 연산에 사용할 3*3 윈도우를 효율적으로 구성할 수 있도록 하였다.

② 동작 원리

입력 스트림은 먼저 1차 버퍼(DIN_buffer)에 저장된다. DIN_buffer와 두 개의 라인 버퍼는 새로운 입력이 들어올 시 규칙에 따라 한 칸씩 시프트되며, 특정 열에 해당하는 3*1 데이터 스트림이 MAC9 유닛으로 전달된다. MAC9 유닛은 이 3*1 스트림을 내부적으로 순차 시프트하여 최종 3*3 윈도우를 구성하고 컨볼루션 연산을 수행한다.

③ 효과

이 구조는 DRAM 접근을 최소화하면서 입력 데이터를 효율적으로 재사용할 수 있으며, 다양한 입력 크기, 스트라이드, 패딩 값에도 동일한 시프트 알고리즘을 적용하며 대응이 가능하다. 결과적으로, Shift Buffer는 본 설계의 유연성과 처리 성능을 동시에 확보할 수 있도록 해주는 핵심 구조이다.

3.3.2 MAC9 병렬 유닛

MAC9 유닛은 Shift Buffer로부터 이미지 데이터를 받아 3*3 윈도우 데이터를 구성하고 BRAM으로부터 필터 데이터를 입력받아 곱셈-누산 연산을 수행하는 블록이다. 각 유닛은 9개의 곱셈기와 누산기로 이루어져 있으며, 이를 통해 한번에 하나의 3*3 필터에 대한 컨볼루션 결과를 계산한다.

① 설계 의도

컨볼루션 연산은 매우 많은 곱셈, 덧셈 연산을 요구하므로 단일 MAC 구조로는 실시간 처리가 어렵다. Arty Z7-20 보드의 DSP 자원이 제한되어 있으므로, 자원 허용량 내에서 병렬성을 극대화하는 구조가 필요했다. 이에 따라 본 연구에서는 MAC9 유닛을 16개씩 병렬 배치하여 한 번의 클럭에서 16개의 출력 채널을 동시에 계산하도록 설계하였다.

② 동작 원리

FSM의 READ_FILTER 상태에서 필터 데이터가 스트리밍을 통해 유입되며 FILTER_BRAM에 저장된다. 각 MAC9 유닛은 FILTER_BRAM으로부터 서로 다른 필터 값을 읽어, 구성해놓은 3*3 윈도우와 곱셈-누산 연산을 수행한다. 연산은 3단 파이프라인 구조로 이루어지며, 곱셈 수행 → 행 단위 부분합 생성 → 전체 합산의 순으로 진행된다. 파이프라인이 완전히 채워진 이후에는 매 클럭마다 16개의 새로운 컨볼루션 결과가 생성된다.

③ 효과

16개의 MAC9 유닛이 동시에 동작함으로써 단일 유닛 대비 약 16배의 연산 속도 향상을 달성하였다. 파이프라인 구조로 인해 여러 출력채널에 대한 컨볼루션 연산이 끊기지 않아 높은 처리율을 유지한다. 동일한 MAC9 모듈을 인스턴스하므로 향후 필요 시 더 많은 병렬 유닛으로 확장할 수 있다.

3.3.3 BRAM 및 FIFO 메모리

BRAM과 FIFO는 각각 필터, 바이어스 저장과 출력 피쳐맵의 누적을 담당한다. 컨볼루션 연산 결과는 매번 MAC9 유닛에서 발생하며, 이를 안정적으로 버퍼링하고 입력 채널별 결과를 순차적으로 합산하기 위해 효율적인 메모리 구조가 필요하다.

① 설계 의도

다중 입력 채널 구조의 CNN 컨볼루션에서는 각 입력 채널마다 독립적인 MAC 연산이 수행되고, 모든 채널의 결과를 더한 뒤 바이어스를 더해야 최종 출력이 완성된다. 즉, 입력 채널 수가 64라면 출력 한 채널의 각 (행,열) 위치마다 64회의 누산이 수행되어야 한다. 직관적으로는 한 채널 컨볼루션의 중간 결과를 BRAM에 저장하고, 다음 채널 컨볼루션 연산을 할때 읽어 누적 후 다시 쓰는 임의 주소 접근 방식이 있다. 그러나 이 방식은 가변적인 크기의 출력 피쳐맵을 모두 호환하기 위해서는 보드의 제한을 뛰어넘는 엄청난 BRAM 사용량이 필요하다는 문제가 있었다.

본 연구에서는 이러한 문제를 해결하기 위해 FIFO 기반의 순차 누적 구조를 도입하였다. 우리의 설계에서 한 입력 채널에 대한 출력 피쳐맵은 매번 (행*열*출력채널16개) 형태의 고정된 구조로 계산된다는 특성이 있으므로, 이를 매 입력 채널마다 반복하며 누적할 때 데이터의 입출력 순서만 보장되는 FIFO 구조를 활용하였다. 이를 통해 임의 주소 접근이 필요한 BRAM 구조 대신, 병렬 MAC 연산 단위에 맞춰 FIFO의 Width를 16*32bit로 고정하고, 데이터 양에 따라 FIFO 깊이(Depth)만 가변적으로 사용하여 다중 채널 환경에서도 효율적인 누적 연산을 구현하였다. 이 방식을 통해 온칩 메모리 사용량을 획기적으로 절감하고, 제어부를 단순화하는 효과를 얻었다.

예를들어, 가변적인 입력 상황에 따라 512*4*4 크기와 64*32*32 크기의 임시 출력을 모두 저장해야 하는 경우가 있을 수 있다. 이때, BRAM/레지스터 방식은 두 경우를 모두 지원하려면 가장 큰 코너 케이스인 512*32*32의 3차원 버퍼를 할당해야 하며, 이때 약 16.8Mbit의 막대한 자원이 소모된다. 하지만 FIFO 방식은 데이터의 총량만큼만 깊이를 할당하면 되므로, 필요한 FIFO의 깊이는 $\max(512/16*4*4, 64/16*32*32)=4096$ 이다. 따라서 한번에 16개의 데이터(16*32bit)를 저장하는 4096 깊이의 FIFO만 있으면 되며, 필요한 메모리는 약 2.1Mbit로 이 경우 자원 사용량을 8배 저감하는 효과가 있다.

이러한 FIFO 기반 누적 구조는 3D 컨볼루션 연산을 2D 컨볼루션의 반복으로 단순화하는 아이디어에 기반한다. 즉, 입력 채널이 여러 개인 복잡한 3차원 컨볼루션을 ‘한 채널씩 순차적으로 2D 컨볼루션을 수행한 뒤 FIFO를 활용하여 누적하는 과정’으로 대체하였다. 이 과정에서 이전 채널까지의 임시 피쳐맵이 FIFO에 임시 저장되고, 다음 채널의 결과가 도착하면 자동으로 누적되며, 마지막 입력 채널이 처리될 때 최종 출력 피쳐맵이 완성된다.

② 동작 원리

필터 및 바이어스 저장 : FSM의 READ_FILTER 상태에서 필터 데이터가 AXI-Stream을 통해 수신되어 FILTER_BRAM에 저장된다. READ_BIAS 상태에서는 바이어스 데이터가 BIAS_BRAM에 기록된다. 두 BRAM은 연산 중에는 읽기 전용으로 동작하며, MAC9 유닛이 주기적으로 필요한 3*3 필터를 병렬로 읽어 사용한다.

연산 결과 누적 : MAC9 유닛으로부터 이번 입력채널의 컨볼루션 결과가 생성될 때, FIFO의 top에는 이전 입력 채널까지 누적된 동일한 행*열*출력채널 위치의 데이터가 존재하게 되며 이는 이번

입력 채널의 컨볼루션 값과 누적해 다시 FIFO의 맨 뒤에 저장된다. 이러한 방식으로 모든 입력 채널에 대한 누적이 순차적으로 이루어지며, 마지막 채널이 끝난 시점에서 FIFO에는 완성된 출력 피쳐맵이 저장된다.

출력 및 전송 : 누적이 완료된 데이터는 바이어스와 더해진 후, FSM의 SEND_OUTPUT 상태에서 AXI-Stream 인터페이스를 통해 DMA로 전송된다. DMA는 이를 DDR 메모리에 저장하고, Python 코드가 결과를 수신하며 이후 과정을 진행한다.

③ 효과

이와 같은 BRAM과 FIFO의 혼용 구조는 메모리 자원 효율성과 제어 단순화 측면에서 뚜렷한 효과를 보였다. 출력 피쳐맵 누적을 FIFO로 처리함으로써 기존 BRAM 기반 구조 대비 온칩 메모리 사용량을 크게 줄였고, 주소 생성 로직이 제거되어 FSM 복잡도 또한 감소하였다. FIFO는 데이터의 입출력 순서를 자동으로 보장하므로 매 입력 채널마다 연산 결과가 안정적으로 누적되며, 깊이만 조정하면 출력 채널 제한 증가에도 쉽게 대응할 수 있다. 결과적으로 본 구조는 BRAM(필터, 바이어스 저장) + FIFO(출력 누적)의 효율적 분리 설계를 통해, 제한된 자원을 가진 Arty Z7-20 보드에서도 다채널 CNN 컨볼루션을 안정적이고 효율적으로 수행할 수 있도록 한 본 연구의 핵심 설계 아이디어라 할 수 있다.

3.3.4 FSM 제어 및 동작

본 연구에서 설계한 FSM은 전체 컨볼루션 연산 과정 제어 및 각 하위 구조(Shift Buffer, MAC9, BRAM 및 FIFO, AXI 인터페이스 등)의 타이밍 제어를 담당한다.

① 설계 의도

각 상태의 역할은 다음과 같다.

- BEFORE_START : Python 드라이버가 start 신호 입력을 보낼때까지 대기한다. 이전 연산에서의 channel_cnt가 초기화되지 않는 문제를 방지하기 위해 이 상태에서 명시적으로 0으로 초기화한다.
- IDLE : 다음 입력 채널의 연산을 시작하기 직전, channel_cnt를 제외한 모든 내부 레지스터와 카운터를 0으로 초기화한다.
- READ_FILTER : AXI-Stream 핸드셰이크를 통해 현재 입력 채널에 필요한 모든 필터 데이터를 FILTER_BRAM에 순차적으로 로드한다.
- RUN_1 : 외부 파라미터와 내부 카운터를 비교하여, 현재 처리할 픽셀이 Stream에서 읽어야 할 유효 데이터인지, 0으로 채워야 할 패딩 영역인지 판단한다.
- RUN_2 : RUN_1에서 결정된 입력값을 din_buffer에 저장하고 시프트 동작을 통해 3*1 스트립을 형성한다. 스트라이드 카운터를 확인하여 컨볼루션 연산을 수행할 타이밍이면 RUN_3로, 아니라면 RUN_1으로 돌아간다.
- RUN_3 : MAC9 내에 완성된 3*3 이미지 윈도우에 대해, BRAM에 저장된 필터를 16개씩 MAC9 유닛에 로드하여 병렬 연산을 수행한다. 파이프라인으로 출력되는 결과는 FIFO에서 읽어온 이전 입력 채널까지의 누적 값과 더해져 다시 FIFO의 맨 뒤에 저장된다.
- READ_BIAS : 모든 컨볼루션 및 누적 연산이 완료된 후, 스트리밍되는 바이어스 데이터를 BIAS_BRAM에 저장한다.
- PRE_SEND_OUTPUT : BRAM/FIFO의 읽기 지연 타이밍 조정을 위한 더미 상태이다.
- SEND_OUTPUT : 모든 연산이 완료된 결과값에 바이어스를 더하며, AXI-Stream을 통해 외부로 전송한다. 데이터를 직렬로 순차적으로 내보내기 위해 시프트 레지스터 로직을 포함한다.
- EOC : 모든 컨볼루션 과정이 완료되었음을 알리고 BEFORE_START 상태로 복귀한다.

결과적으로 본 FSM 구조는 데이터의 입력부터 연산 및 출력까지 이어지는 전 컨볼루션 과정의 흐름을 단계별로 분리하여 제어하며, 초기 연산 시작 신호 이후의 모든 과정을 하드웨어 단독으로 자동 수행할 수 있도록 하였다.

3.3.5 가변 파라미터에 대응하기 위한 구조적 설계 아이디어

본 연구의 Convolution IP는 단일 하드웨어 구조로 입력 이미지 크기, 패딩, 스트라이드, 입출력 채널 수 등 다양한 파라미터 조합에 대응할 수 있도록 설계되었다. 이러한 준-범용성을 확보하기 위해, 본 연구에서는 하드웨어의 물리적 배선을 고정한 상태에서 FSM 제어와 내부 카운터 동작만으로 파라미터 변화에 대응하는 접근 방식을 채택하였다. 즉, 즉, 하나의 하드웨어 구조 내에서 카운터 신호(`img_row_cnt`, `img_col_cnt`, `str_row_cnt`, `str_col_cnt` 등)를 기반으로 상황에 맞게 동작하며, 이를 통해 다양한 컨볼루션 연산을 수행할 수 있다.

(1) 입력 이미지 크기에 대한 대응

입력 이미지의 크기가 1*1부터 32*32까지 가변적일 수 있는 환경에서도 동일한 하드웨어로 처리할 수 있도록, 라인 버퍼 기반 Shift Buffer 구조를 적용하였다. Shift Buffer는 항상 고정된 형태의 3*1 스트림을 MAC9 유닛으로 공급하며, 입력 폭이 달라지더라도 FSM 제어부의 `IMAGE_WIDTH`와 `PADDING`값에 따라 버퍼내에 임시 저장된 데이터의 시프팅 경로만 달라진다.

즉, 여러 이미지 크기에 따른 물리적 배선은 모두 존재하되, 이미지 폭과 패딩 조합에 따라 시프팅 과정에서 ‘왼쪽 끝 데이터가 오른쪽 어느 열로 재진입할지’를 결정하는 다중 경로(MUX)가 합성 시 생성되며, 실행 시에는 입력 파라미터에 맞는 경로만 활성화된다. 이 구조 덕분에 가변적인 입력이 주어지더라도 Shift Buffer는 유사한 방식으로 동작하며, 매 입력마다 저장된 데이터를 시프트하면서 버퍼를 갱신하고, `{shift_buffer[0][2], shift_buffer[1][2], din_buffer}`의 고정된 위치 데이터만 MAC9 유닛으로 전달한다.

실제 입력 이미지 데이터는 버퍼에서 시프팅되며 재사용을 필요로 하는 일부만 임시 저장되지만, 현재 처리 중인 좌표는 FSM의 `img_row_cnt`와 `img_col_cnt` 카운터로 관리되며, 이들은 실제 하드웨어 상의 버퍼가 아닌 ‘가상의 2차원 좌표계’를 의미한다. 예를 들어 32*32 입력에 패딩이 1이라면, FSM은 34*34 크기의 가상 입력 맵에서 (0,0)부터 (33,33)까지 순차적으로 탐색하며 현재 좌표가 실제 입력 이미지 영역이어야 하는지, 가장자리의 패딩 영역이어야 하는지를 구분한다. 이러한 좌표 기반 제어 방식 덕분에 입력 크기와 패딩이 달라져도 간단한 비교기만으로 모든 경우를 쉽게 처리할 수 있다.

(2) 패딩과 스트라이드에 대한 대응

패딩은 FSM의 `RUN_1` 상태에서 수행된다. `img_row_cnt`와 `img_col_cnt`가 현재 처리 중인 좌표를 나타내며, 이 값이 패딩 범위 안에 속하면 해당 입력은 제로 패딩으로 처리된다. 즉, 현재 입력 데이터가 가장자리 영역에 위치할 경우 시프트 버퍼에 0이 저장되고, 그렇지 않은 경우 외부 스트림으로부터 실제 이미지가 로드되어 저장된다.

스트라이드는 FSM의 `RUN_2` 상태에서 제어된다. `str_row_cnt`와 `str_col_cnt`는 각각 행과 열 방향의 스트라이드 이동을 추적하는 카운터로, `(str_row_cnt, str_col_cnt) == (0, 0)`일 때만 FSM이 `RUN_3` 상태로 진입하여 컨볼루션 연산을 수행한다. 예를 들어 스트라이드가 3이면, 카운터는 0, 1, 2, 0, 1 ... 순으로 반복되며, 매 세 번째 위치에서만 MAC9 유닛이 동작한다. 이는 복잡한 주소 계산이나 로직 없이 단순한 카운터 비교로 모든 스트라이드 조합을 처리할 수 있게 하며, FSM 전체 구조의 단순성과 범용성을 동시에 유지할 수 있는 아이디어이다.

(3) 입력 및 출력 채널에 대한 대응

입력 채널(`INPUT_CHANNEL`)이 여러 개인 경우, 각 채널의 컨볼루션 결과는 FIFO를 통해 순차적으로 누적된다. 즉, 하나의 입력 채널에 대한 컨볼루션이 완료될 때마다 이전 채널까지의 누적값

과 더한 후 다시 FIFO에 저장하는 구조이다. 이 과정을 모든 입력 채널에 대해 반복하면, 다채널 입력에 대한 최종 누적 결과가 완성된다.

출력 채널(OUTPUT_CHANNEL)의 가변성은 16개의 병렬 MAC9 유닛이 동시에 16개의 필터를 병렬 계산하는 과정에서 파이프라인 통해 전체 필터를 다 사용할 때까지 순차적으로 반복 처리하는 방식으로 대응하였다. used_filter 카운터가 현재까지 계산된 필터 수를 추적하며, 이 값이 AXI-Lite로 설정된 OUTPUT_CHANNEL 값에 도달하면 FSM은 자동으로 다음 단계로 전환된다.

(4) 정리 및 의의

결과적으로, 본 구조는 입력 크기, 패딩, 스트라이드, 채널 수 등 모든 가변 하이퍼 파라미터를 Shift Buffer 및 FSM의 카운터의 동작으로 대응한다. 하드웨어 구조는 항상 동일하지만, FSM의 논리 제어와 카운터 동작이 실시간으로 이미지 좌표, 패딩 삽입 여부, 스트라이드 주기, 처리 채널 수를 판단하여 전체 컨볼루션 연산을 정확히 수행한다. 즉, 하드웨어의 구조적 복잡도를 크게 늘리지 않고도 다양한 CNN 컨볼루션 연산 조합에 자동 대응하는 효율적인 아키텍처라 할 수 있다.

4. IP 생성 및 SoC 통합

앞선 절에서 설계된 컨볼루션 가속기의 RTL은 Vivado 환경에서 AXI 인터페이스 기반 사용자 정의 IP로 패키징하여, Zynq-7000 SoC(Arty Z7-20) 상에 PS와 PL이 통합된 형태로 구현하였다. 본 절에서는 IP 생성부터 Block Design 구성, 그리고 최종 비트스트림 생성까지의 과정을 간략히 설명한다.

(1) IP 생성

작성된 Verilog 모듈(convolution.sv, MAC9.sv)은 Vivado의 IP Packager를 이용하여 AXI-Lite 및 AXI-Stream 인터페이스를 동시에 지원하는 독립적인 IP로 생성하였다.

AXI-Lite 인터페이스는 PS(CPU)가 CNN 컨볼루션 연산 파라미터(IMAGE_WIDTH, CHANNEL, STRIDE, PADDING 등)를 설정하고, 연산 시작 및 종료 신호를 제어하기 위한 경로로 사용된다. Python(PYNQ) 코드에서 직접 IP 내부의 제어 레지스터(0x00~0x18)를 접근함으로써 이를 기반으로 하드웨어 연산을 수행한다.

AXI-Stream 인터페이스는 DMA와 Convolution IP 간의 고속 데이터 전송 통로로 사용되며, 입력, 필터 및 바이어스, 출력 피쳐맵 데이터가 스트리밍 방식으로 전송된다. TVALID/TREADY 핸드셰이크를 구현하여, 내부가 연산 중일 때 입력 스트림을 일시적으로 정지시켜 데이터 유실 없이 안정적인 연산을 보장한다.

(2) 블록 디자인 구성

패키징된 Convolution IP는 Vivado의 Block Design 환경에서 Zynq7 Processing System, 3개의 AXI DMA(입력, 필터 및 바이어스, 출력 용), SmartConnect와 함께 결합되었으며 PL의 클럭은 100 MHz로 설정하였다.

Zynq7 PS는 상위 제어를 담당하며, Python(PYNQ) 환경에서 AXI-Lite 인터페이스를 통해 IP의 레지스터를 설정한다.

3개의 DMA는 각각 입력 이미지, 필터 및 바이어스, 출력 데이터를 담당하여 병렬적인 스트리밍 전송을 가능하게 한다.

SmartConnect는 PS, DMA, Convolution IP 간의 AXI 버스 연결을 자동으로 구성한다.

(3) 합성, 구현 및 비트스트림 생성

Block Design 구성이 완료된 후, Vivado의 Synthesis와 Implementation 단계를 거쳐 타이밍 여유 및 리소스 사용량을 분석하였다. 이후 비트스트림을 생성하고 Zynq 보드에 업로드하여 Python(PYNQ) 환경에서 DMA를 이용한 데이터 송수신 및 Convolution IP 제어가 정상적으로 수행됨을 확인하였다. 이 과정을 통해 우리의 설계가 실제 SoC 시스템 내에서도 완전한 동작을 수행함을 확인하였다.

5. Vivado 합성 및 자원 사용 결과

본 연구에서 설계한 Convolution IP는 Vivado 2022.2 환경에서 Zynq-7000 SoC(Arty Z7-20, XC7Z020-1CLG400C)를 대상으로 합성(Synthesis) 및 구현(Implementation)을 수행하였다. 합성 및 구현 결과, 전체 구조는 보드의 자원 한계 내에서 안정적으로 구현되었으며, 세부 자원 사용량은 다음 표와 같다.

항목	사용량	전체 대비 사용량(%)
LUT	9399	17.67%
LUTRAM	849	4.88%
FF	10634	9.99%
BRAM	84	60.00%
DSP	144	65.45%

요약 및 분석

전체 설계는 약 17.67%의 LUT, 4.88%의 LUTRAM, 9.99%의 FF를 사용하였으며, FIFO 기반 누적 구조를 적용하여 BRAM 사용량을 크게 절감하였다. MAC9 병렬 유닛은 DSP 자원 한계 내에서 16채널 동시 연산이 가능하도록 구성되었다. 동작 주파수는 100 MHz, 타이밍 여유(WNS)는 0.274 ns로 나타나, 전체 설계가 100 MHz 동작 환경에서 안정적으로 수행됨을 확인하였다. Dynamic Power는 1.513W, Static Power는 0.149W로 Total On-chip Power는 1.662W로 추정되었다.

IV. 실험 및 결과

1. 실험 개요

본 연구에서 제안한 컨볼루션 가속기 IP는 Vivado 및 PYNQ 환경에서 단계적으로 검증되었으며, 실험은 다음 네 단계로 수행하였다.

1. RTL 시뮬레이션

Vivado 환경에서 Testbench를 통해 RTL 레벨에서 설계자의 의도대로 컨볼루션 연산이 정상 동작하는지 확인하였다.

2. 기능 및 성능 검증

SoC 환경에서 랜덤 정수형 데이터를 이용하여 FPGA 기반 연산의 정확성과 CPU(for-loop) 대비 연산 속도 향상을 검증하였다.

3. 양자화 기반 Simple 모델 검증

임의로 Simple VGG 모델을 구성해 CIFAR-10 데이터 셋을 학습 시키고, 추론 과정에서 컨볼루션 레이어 연산을 FPGA 가속기로 대체하여 전 과정 PyTorch 추론 결과 대비 분류 정확도 및 처리속도를 비교하였다.

4. 양자화 기반 상용 모델 검증

실험을 위해 임의로 구성한 모델이 아닌, 실제 Torchvision 라이브러리에서 제공하는 VGG-16 모델을 바탕으로 파인 튜닝을 거쳐 (3)과 동일한 실험을 진행하였다.

2. 실험 환경

항목	내용
보드	Digilent Arty Z7-20 (Zynq XC7Z020-1CLG400C)
프로세서	ARM Cortex-A9 (Dual-core, 650 MHz)
소프트웨어	Vivado 2022.1 / PYNQ 2.7 (Jupyter Notebook)
통신 인터페이스	AXI4-Lite / AXI4-Stream
클럭 주파수	PL: 100 MHz
비교 기준	CPU 단독 연산 vs FPGA 연산 (DMA 전송 포함)

3. 실험 1 – RTL 시뮬레이션 기초 동작 검증

① 목적

IP 패키징 이전 단계에서 Vivado 시뮬레이터와 Testbench를 이용하여 컨볼루션 가속기의 기본 동작을 RTL 수준에서 검증하였다. 이는 설계 과정 중 각 모듈의 논리적 정합성과 데이터 흐름을 점검하기 위한 기초 검증 단계이다.

② 구성 및 절차

1. Vivado에서 검증을 위한 Testbench를 구성하고, 클럭, 리셋 신호 및 임의의 입력 데이터를 인가하였다.
2. FSM 상태 전이, 데이터 시프트 타이밍, 연산기 출력 결과 등을 비교, 관찰하였다.
3. RTL 코드 수정 및 내부 모듈 조정 시마다 동일한 방식으로 반복 시뮬레이션을 수행하여, 설계 변경이 의도대로 반영되는지를 확인하였다.

③ 분석

본 단계는 특정 실험 결과를 분석하기 보다는, 설계 및 수정 과정 전반에서 컨볼루션 IP의 내부 동작이 정상적으로 이루어지는지를 주기적으로 확인하는 데 목적이 있었다. 이 과정을 통해 설계한 모듈의 타이밍 및 로직이 의도적으로 동작함을 반복적으로 검증하였으며, SoC 통합 전 충분한 디버깅을 거칠 수 있었다.

4. 실험 2 – 랜덤 정수 데이터 기반 기능 및 성능 검증

① 목적

FPGA에서 구현된 Convolution IP의 기능적 정확성과 연산 성능 향상 효과를 검증한다. 본 실험은 임의의 정수형 입력 데이터를 사용하여, 연산 구조의 기능적 일치 여부와 CPU 대비 처리속도를 평가하였다.

② 구성 및 절차

1. Python 환경에서 입력 이미지, 필터, 바이어스 데이터를 NumPy를 사용해 임의의 정수(입력 및 필터 INT8, 바이어스 INT32)로 생성하였다.
2. 동일한 데이터 셋으로 CPU(for-loop)와 FPGA(Convolution IP)에서 각각 컨볼루션을 수행하였다.
3. 두 결과를 비교하여 일치 여부를 확인하였다.
4. 각 연산의 수행 시간을 측정하여, CPU 대비 FPGA의 가속 비율을 계산하였다.

③ 실험 변수

변수 항목	값
입력 크기	8×8, 16×16, 32×32
입력 채널 수	1, 3, 16
출력 채널 수	8, 16, 32
패딩 (padding)	0, 1, 2
스트라이드 (stride)	1, 3, 5

④ 결과 요약

입력크기	입력채널	출력채널	패딩	스트라이드	CPU(ms)	FPGA(ms)	시간비	결과일치
8×8	1	8	0	1	45.409	1.6263	27.92	O
16×16	3	16	1	3	172.325	1.6541	104.18	O
32×32	16	32	2	5	2126.789	2.3093	920.97	O

모든 테스트 케이스에서 FPGA 연산 결과는 CPU 결과와 일치하였으며, 이는 우리의 IP 내부 연산이 정상적으로 동작했음을 의미한다. 입력 크기 및 채널 수가 증가할수록 FPGA의 병렬 구조 이점이 뚜렷해졌으며, 지정한 테스트 케이스의 경우 CPU 대비 최대 수백 배의 속도 향상을 확인하였다.

⑤ 분석

FPGA에서는 MAC9 병렬 구조와 파이프라인 처리로 인해, CPU의 순차 연산(for-loop) 대비 현저히 높은 처리율을 확보하였다. DMA 데이터 전송 오버헤드가 있음에도 불구하고, FPGA의 연산 성능이 CPU보다 우수하였다. 본 실험을 통해 IP의 기능적 정확성과 성능적 우수성이 모두 입증되었다.

5. 실험 3 – 양자화 기반 Simple 모델 추론 검증

① 목적

학습된 CNN 모델의 실제 추론 과정에 양자화 및 FPGA 가속기를 적용하고, PyTorch의 부동소수점 연산 결과와 비교하여 정확도와 처리속도 면에서 유의미한 성능을 제공함을 검증한다.

② 구성 및 절차

1. 모델 학습 및 .pth 파일 추출

CIFAR-10 데이터셋을 이용하여 Simple VGG 모델을 PyTorch로 학습하였다.

학습 완료 후 .pth 가중치 파일을 추출하였다.

2. 양자화 적용

FPGA 가속기의 입력 규칙에 맞춰 컨볼루션 레이어의 입력, 필터, 바이어스를 각각 INT8, INT8, INT32 형식으로 변환하였다.

3. 하이브리드 추론 수행

(a) Baseline : 전 과정 PyTorch 부동소수점 연산으로 추론을 진행한다.

(b) FPGA Hybrid : PyTorch 추론 중 컨볼루션 레이어만 FPGA 가속기로 대체한다,

- Python 코드에서 DMA를 통해 입력과 필터 및 바이어스를 FPGA로 전송 후 연산 수행

- FPGA 결과를 다시 CPU로 수신하여 역양자화 후 후속 레이어 추론을 계속 진행

4. 결과 비교

분류 정확도와 평균 추론 시간(1,000개 이미지 평균)을 비교하였다.

③ 결과 요약

구분	추론 정확도(%)	평균 추론 시간(ms)	시간비
PyTorch	80.7%	3314.74	
Hybrid (Conv IP + PyTorch)	80.6%	260.89	12.71

실험 결과, 컨볼루션 레이어의 하드웨어 가속만으로도 CNN 전체 추론 병목을 크게 개선할 수 있음을 확인 할 수 있었다. 양자화 및 하이브리드 추론의 정확도 손실은 약 0.1%p 이내로 미미하였으며, 하이브리드 추론은 평균 처리 속도를 약 12.71배 향상시켰다.

④ 분석

정수형 양자화 기반 추론은 경량 FPGA 가속기와의 호환성을 확보하면서도 부동소수점 대비 거의 동일한 정확도를 유지하였다. 본 실험을 통해 우리가 제안한 CNN 컨볼루션 IP가 실제 추론 환경에서도 정확도 유지 및 처리속도 향상을 모두 만족함을 입증하였다.

6. 실험 4 – 양자화 기반 상용 모델 추론 검증

① 목적

임의로 정의한 Simple 모델이 아닌, Torchvision에서 제공하는 VGG16 상용 모델을 기반으로 파인튜닝한 실제 추론과정에 양자화 및 FPGA 가속기를 적용하고, Pytorch 부동소수점 연산 결과와 비교하여 정확도와 처리속도 면에서 유의미한 성능을 제공함을 검증한다. 본 실험의 상용 모델로 VGG16을 선택한 이유는 모델의 모든 컨볼루션 레이어가 3*3 필터로 통일되어 있어, 본 연구에서 설계한 가속기의 성능과 실효성을 검증하기에 가장 적합하다고 판단하였기 때문이다.

② 구성 및 절차

1. 모델 학습 및 .pth 파일 추출

Torchvision에서 ImageNet으로 사전 학습된 VGG16 모델을 로드하고, CIFAR-10 데이터셋에 맞게 파인튜닝한다. 이때, CIFAR-10 이미지는 입력 크기가 32*32이므로, 레이어를 통과하는 과정에서 입력이 과하게 작아지는 것을 방지하기 위해 3개의 MaxPool 레이어를 제거하고, DRAM 제한을 고려하여 classifier의 FC 레이어를 1024차원으로 축소하는 방향으로 미세 조정 후 학습하였다.

2. 양자화 적용

FPGA 가속기의 입력 규칙에 맞춰 컨볼루션 레이어의 입력, 필터, 바이어스를 각각 INT8, INT8, INT32 형식으로 변환하였다.

3. 하이브리드 추론 수행

(a) Baseline : 전 과정 PyTorch 부동소수점 연산으로 추론을 진행한다.

(b) FPGA Hybrid : PyTorch 추론 중 컨볼루션 레이어만 FPGA 가속기로 대체한다,

- Python 코드에서 DMA를 통해 입력과 필터 및 바이어스를 FPGA로 전송 후 연산 수행
- FPGA 결과를 다시 CPU로 수신하여 역양자화 후 후속 레이어 추론을 계속 진행

4. 결과 비교

분류 정확도와 평균 추론 시간(100개 이미지 평균)을 비교하였다.

③ 결과 요약

구분	추론 정확도(%)	평균 추론 시간(ms)	시간비
PyTorch	83.0%	23223.78	
Hybrid (Conv IP + PyTorch)	81.0%	1375.73	16.88

실험 결과, 컨볼루션 레이어의 하드웨어 가속만으로도 CNN 전체 추론 병목을 크게 개선할 수 있음을 확인 할 수 있었다. 양자화 및 하이브리드 추론의 정확도 손실은 약 2.0%p이내로 미미하였으며, 하이브리드 추론은 평균 처리 속도를 약 16.88배 향상시켰다.

④ 분석

정수형 양자화 기반 추론은 경량 FPGA 가속기와 호환성을 확보하면서도 부동소수점 대비 거의 동일한 정확도를 유지하였다. 본 실험을 통해 우리가 제안한 CNN 컨볼루션 IP가 실제 상용 모델 추론 환경에서도 정확도 유지 및 처리속도 향상을 모두 만족함을 입증하였다.

본 실험에서 전체 10,000개의 테스트 이미지가 아닌 100개의 샘플 이미지만을 테스트에 사용한 이유는, Baseline의 성능을 측정한 Arty Z7-20 보드의 내장 CPU의 성능 한계 때문이다. 측정 결과, 이미지 한 장당 추론 시간은 약 23초 정도 소요되며 전체 테스트셋을 검증하는 것은 현실적으로 불

가능했다. 이러한 CPU의 극심한 성능 한계는 임베디드 환경에서의 CNN 컨볼루션 연산이 심각한 병목을 유발함을 명확히 보여주는 사례로, 본 실험에서 달성한 16.88배의 추론 속도 향상 결과가 실제 추론 환경에서 큰 의미를 지닌다고 볼 수 있다.

V. 결론

본 연구에서는 Zynq-7000 SoC(Arty Z7-20) 기반 FPGA 상에서 CNN의 핵심 연산인 컨볼루션을 효율적으로 수행하기 위한 준-범용 CNN Convolution IP를 설계하고, 정수형 양자화 기반의 HW/SW 통합 구조로 구현하였다. 제안된 가속기는 입력 크기, 채널 수, 패딩, 스트라이드의 조합이 가변적인 다양한 CNN 연산에 대응할 수 있도록 설계된 준-범용 구조로, 특정 네트워크 모델에 종속되지 않으면서도 임베디드 환경에서 실시간 처리를 가능하게 하는 것을 목표로 하였다.

하드웨어 구조는 CNN에서 가장 널리 사용되는 3*3 커널 전용으로 최적화되어 있으며, Shift Buffer 기반 입력 윈도우 구성을 통해 입력 데이터를 효율적으로 재사용하도록 하였다. 또한 16개의 병렬 MAC9 유닛을 배치하여 높은 연산 병렬성을 확보하였고, BRAM과 FIFO를 혼용한 메모리 구조를 적용함으로써 필터, 바이어스 저장과 출력 피쳐맵 누적을 효율적으로 처리하였다. 특히 FIFO 기반 누적 구조를 도입하여 다채널 누산 과정에서 불필요한 주소 접근을 최소화하였고, 제어 로직의 단순화와 높은 자원 효율성을 달성하였다. 전체 연산의 순서와 데이터 흐름은 FSM을 통해 자동으로 제어되며, 입력, 연산, 출력 전 과정이 하드웨어 단독으로 수행되도록 구성하였다. 또한 Vivado IP Packager를 이용한 AXI4-Lite/Stream 표준 IP 패키징으로 재사용성과 확장성을 확보하였다.

소프트웨어 측면에서는 PYNQ 기반 Python 드라이버를 설계하여 CNN 파라미터(입력 크기, 채널 수, 스트라이드, 패딩 등)를 AXI-Lite 레지스터를 통해 직접 설정할 수 있도록 하였으며, DMA를 이용해 입력, 필터, 출력 데이터를 AXI-Stream 방식으로 전송함으로써 PS와 PL 간의 데이터 흐름을 완전하게 통합하였다. 이를 통해 Python 코드가 단순한 테스트 도구를 넘어 하드웨어 동작을 직접 제어하는 드라이버 역할을 수행하도록 하였다.

실험 결과, 랜덤 정수형 입력 데이터를 이용한 기능 검증에서는 FPGA의 연산 결과가 CPU(for-loop) 기반 연산 결과와 완벽히 일치하였으며, 동일한 입력 조건에서 CPU 대비 최대 수백배의 연산 속도 향상을 확인하였다. 또한, CIFAR-10 데이터셋으로 학습된 Simple VGG 모델을 활용한 양자화 기반 추론 실험에서는 PyTorch 부동소수점 추론 대비 정확도 손실이 약 0.1%p 이내로 매우 작았고, FPGA 가속기를 적용한 경우 평균 12.71배의 추론 속도 향상이 나타났다.

나아가, 실제 상용 VGG16 모델을 파인튜닝하여 동일한 방식으로 비교 검증한 실험에서도, 정수형 양자화로 인한 정확도 하락을 2.0%p 수준으로 억제하면서 평균 16.88배의 추론 속도 향상을 달성하였다. 이는 제안된 하드웨어 구조가 연산 효율성을 유지하면서도 실질적인 정확도를 확보할 수 있으며, 실제 상용 CNN 모델 가속 추론에 문제없이 적용될 수 있음을 의미한다.

결과적으로 본 연구는 3*3 커널 기반 CNN 컨볼루션 연산을 임베디드 FPGA 환경에서 실시간으로 처리할 수 있는 준-범용 구조의 CNN 하드웨어 가속기를 성공적으로 구현한 것이다. 정수형 양자화와 병렬 MAC 구조, 효율적인 메모리 관리, Python 기반 상위 제어를 결합함으로써 CNN 연산의 핵심 병목을 완화하였고, FPGA의 자원 한계 내에서도 안정적 동작을 달성하였다.

우리의 연구는 향후 Batch Normalization, ReLU 등 추가 연산 모듈을 포함한 복잡한 CNN 가속기를 설계할 때, 본 IP를 별도의 수정 없이 컨볼루션 연산 모듈로 인스턴스화하여 활용할 수 있다는 점에서 의의가 있다. 또한 본 설계를 Zynq UltraScale+ 등 상위급 FPGA 플랫폼으로 이식할 경우, 더 높은 동작 주파수와 연산 자원 활용을 통해 전체 연산 성능을 한층 향상시킬 수 있을 것이다.

결과적으로 본 IP는 다양한 네트워크 아키텍처에서 공통적으로 적용 가능한 핵심 모듈로서, 향후 CNN 가속기 설계의 효율성과 접근성을 높이는 데 기여할 수 있을 것으로 기대된다.

※ 참고문헌

- [1] A. Khan, A. Sohail, U. Zahoora, and A. S. Qureshi, "A Survey of the Recent Architectures of Deep Convolutional Neural Networks", Artificial Intelligence Review. DOI: 10.1007/s10462-020-09825-6.
- [2] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition" in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.
- [3] J. Bhasker, "Verilog 합성: 최적의 합성을 위한 설계 가이드", 서울, 대한민국: 홍릉과학출판사, 2010.
- [4] 우영준, "Verilog HDL design using vivado", 서울, 대한민국: 이니프로, 2018.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.